



Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
| Wydział Inżynierii Mechanicznej i Robotyki | Katedra Robotyki i Mechatroniki

INŻYNIERIA MECHATRONICZNA | I stopień | semestr VI | 2025/2026

Programowanie Obiektowe i Inżynieria Oprogramowania

Instrukcja C++/CLI: 4. Wariacje na temat wykresów

Wprowadzenie do aplikacji okienkowych w C++

Nauczysz się:

Pisać kod pod aplikacje okienkowe w języku C++/CLI z wykorzystaniem platformy *.NET framework* w środowisku *Visual Studio*:

- tworzyć wykresy,
- pracować z *timerem* aplikacji,
- generować liczby pseudolosowe,
- łączyć z urządzeniami przez port szeregowy.

Dodatkowe materiały:

- materiały do zajęć: [\[link\]](#),
- oficjalna strona C++/CLI [\[link\]](#),

Kierunkowe efekty kształcenia pokryte w tej instrukcji:
IME1A_U1, IME1A_U4

Koordynator przedmiotu:

dr inż. Lucjan Miękina, miekina@agh.edu.pl

Autor instrukcji:

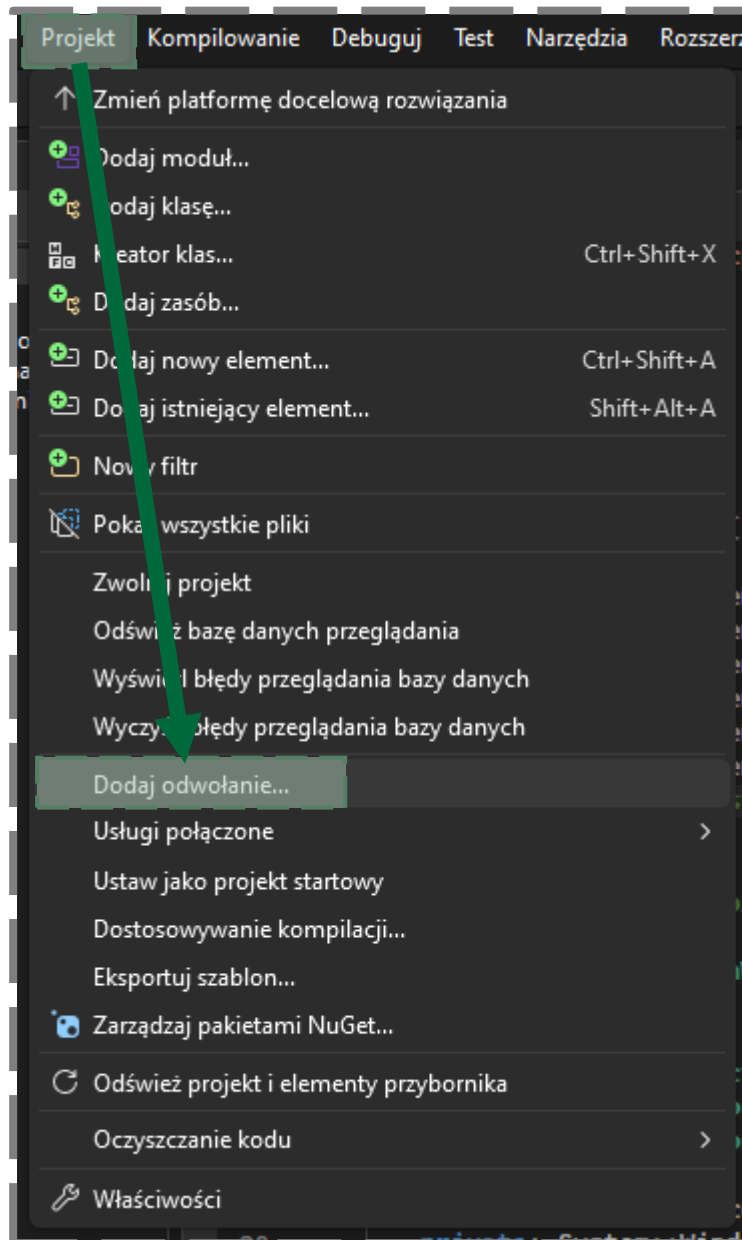
dr inż. Jakub Bryła, jakub.bryla@agh.edu.pl

1. Wprowadzenie

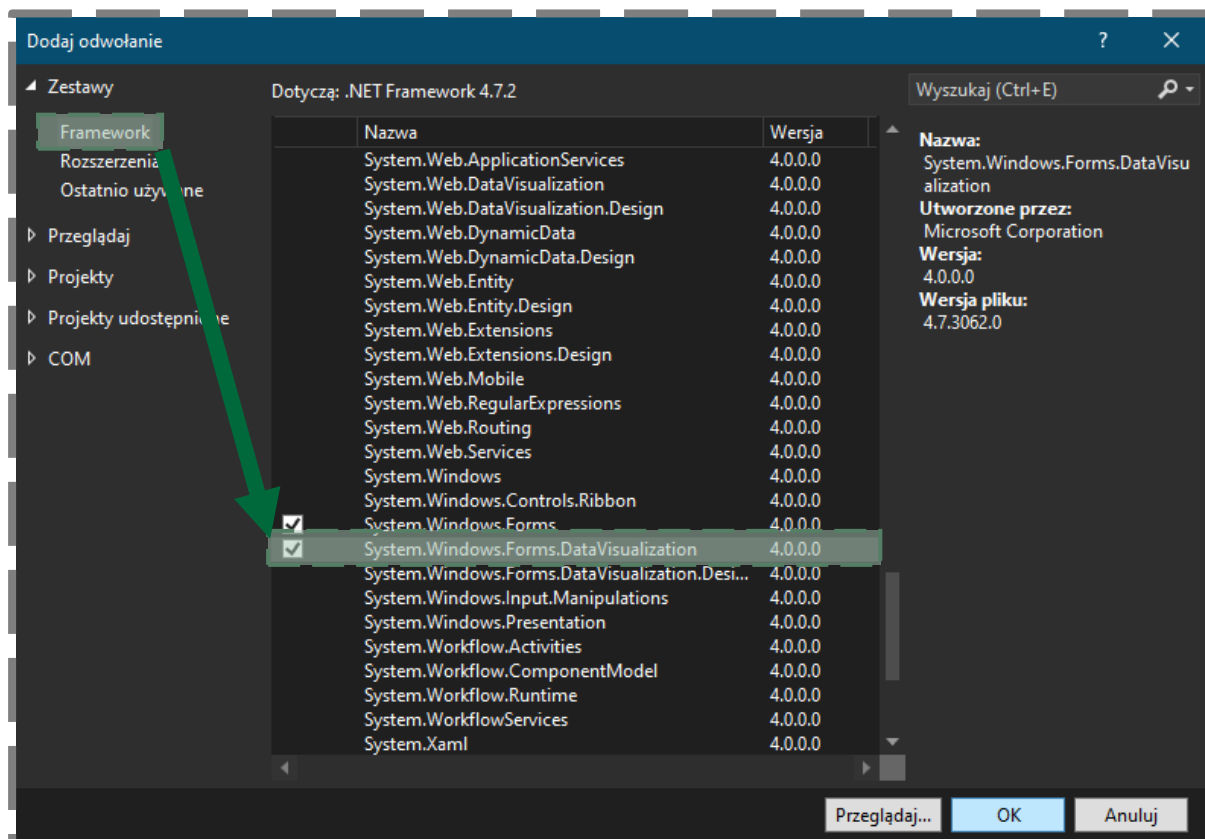
Niniejsza instrukcja skupia się na możliwości tworzenia wykresów w języku C++/CLI:

- “statycznych”, wyświetlających dane programu, wartości zmiennych,
- “dynamicznych”, odświeżanych z zadaną częstotliwością,
- z urządzenia zewnętrznego, komunikującego się z aplikacją po porcie szeregowym.

Dla każdego tworzonego wykresu należy dokonać odpowiednich ustawień projektu:



W nowo otwartym oknie należy przejść do *Zestawy->Framework* i zaznaczyć opcję *System.Windows.Forms.DataVisualization*:

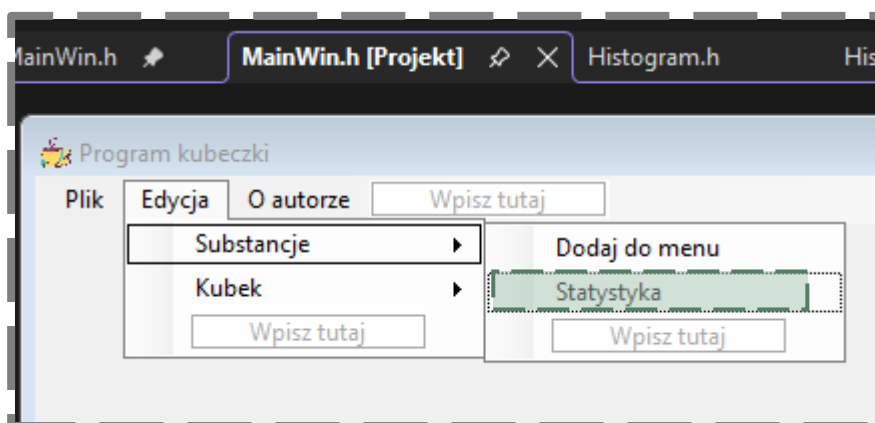


2. Zadania

ZADANIE 2.1 – histogram substancji w kubkach

W pierwszej kolejności należy utworzyć nową *Formę*, w której wyświetlany będzie histogram zużycia substancji. W tym celu kliknij PPM na nazwę projektu w jego drzewku i wybierz opcję *Dodaj->Nowy element...* W zakładce *Interfejs użytkownika* wybierz *Formularz systemu Windows* i wprowadź nazwę „*Histogram.h*”.

Kliknij dwukrotnie LPM na opcję menu „Statystyka”:



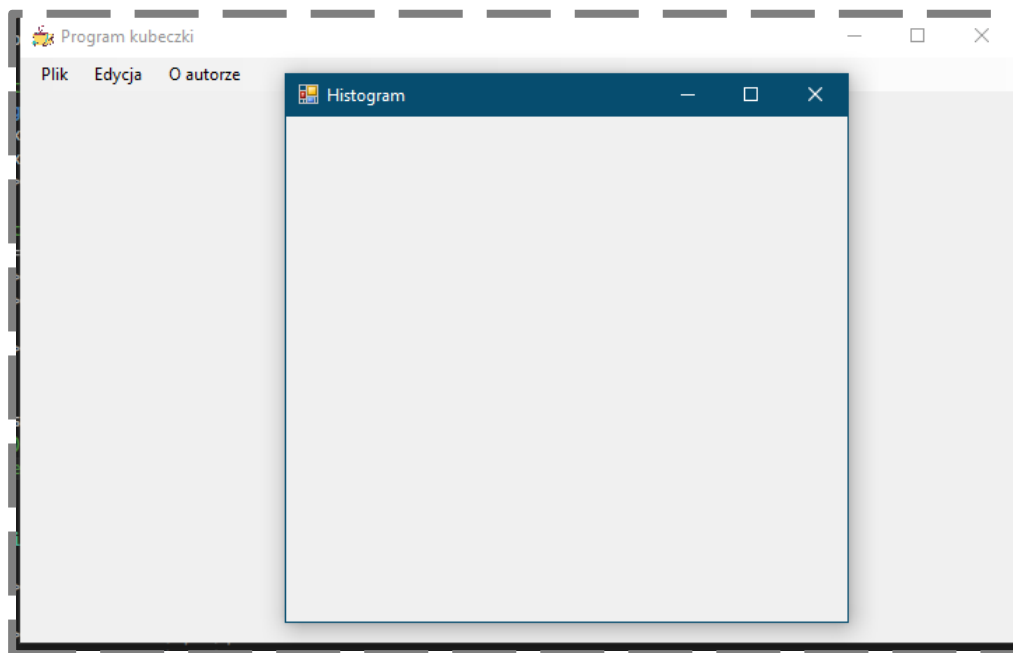
Zostaniesz przekierowany do wygenerowanej funkcji obsługującej zdarzenie kliknięcia na opcję menu. Wprowadź kod:

```
1 private: System::Void statystykaToolStripMenuItem_Click(System::Object^  
2 sender, System::EventArgs^ e) {  
3     Histogram^ hist = gcnew Histogram();  
4     hist->Show();  
5 }  
6
```

Na początku pliku załącz plik „Histogram.h”:

```
1 #include <vector>  
2 #include "OAutarze.h"  
3 #include "Histogram.h"  
4 #include "TCup.h"  
5
```

Po skompilowaniu programu powinieneś zobaczyć nowe okno po wybraniu opcji w menu „Statystyka”:



Spróbujmy utworzyć pierwszy wykres. U góry pliku *Histogram.h* dodaj polecenie `using namespace` oraz zdefiniuj zmienne potrzebne do wygenerowania wykresu:

```
1 using namespace System::Windows::Forms;  
2 using namespace System::Data;  
3 using namespace System::Drawing;  
4 using namespace System::Windows::Forms::DataVisualization::Charting;  
5  
6 /// <summary>  
7 /// Podsumowanie informacji o Histogram  
8 /// </summary>  
9 public ref class Histogram : public System::Windows::Forms::Form  
10 {  
11 private:  
12     Chart^ chart1;  
13     ChartArea^ area;  
14     Series^ series;  
15  
16 public:  
17
```

U dołu pliku zdefiniuj funkcję inicjującą wykres:

```
1 #pragma endregion
2     private: Void init_chart()
3     {
4         // create chart
5         chart1 = gcnew Chart();
6         chart1->Dock = DockStyle::Fill;
7         this->Controls->Add(chart1);
8
9         // create chart area
10        area = gcnew ChartArea("MainArea");
11        area->AxisX->Title = "Substancje [-]";
12        area->AxisY->Title = "Zużyta objętość w kubkach [ml]";
13        chart1->ChartAreas->Add(area);
14
15        // create data series
16        series = gcnew Series("Dane");
17        series->ChartType = SeriesChartType::Column;
18        series->ChartArea = "MainArea";
19
20        chart1->Series->Add(series);
21    }
22
```

Wywołaj funkcję w konstruktorze:

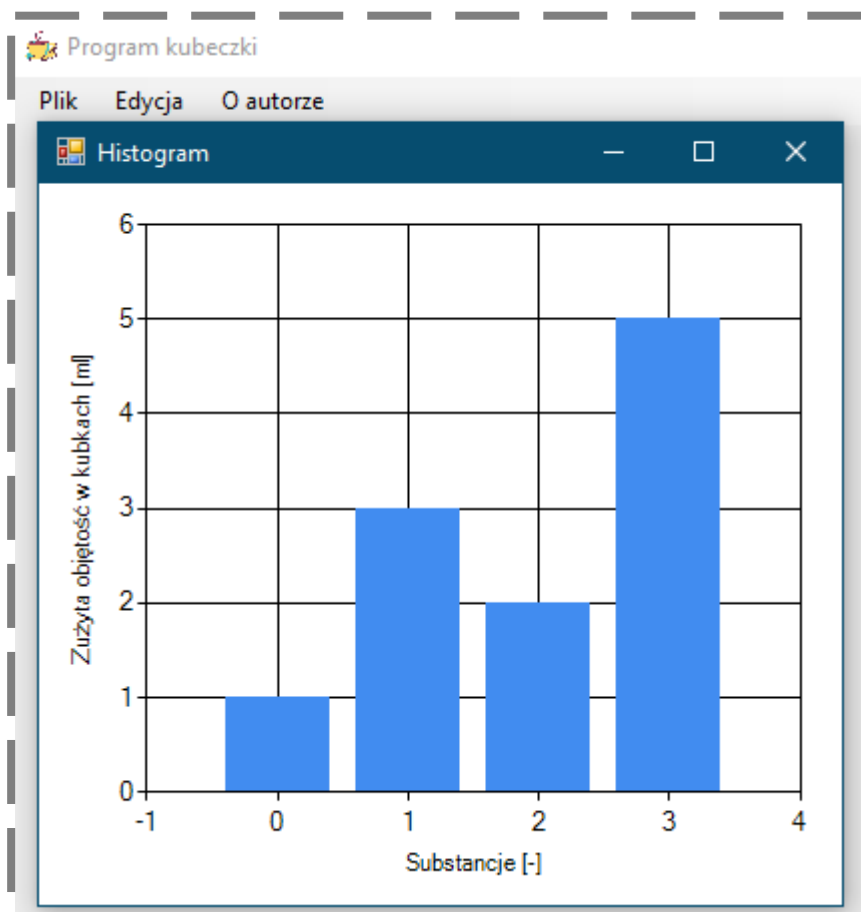
```
1 public ref class Histogram : public System::Windows::Forms::Form
2 {
3 private:
4     Chart^ chart1;
5     ChartArea^ area;
6     Series^ series;
7
8 public:
9     Histogram(void)
10    {
11        InitializeComponent();
12        init_chart();
13    }
14
```

W IDE w zakładce „Histogram.h [Projekt]” kliknij dwukrotnie LPM na tło aplikacji. Zostanie wygenerowana funkcja `Histogram_Load` uruchamiana automatycznie po otwarciu okna:

```
1 private: System::Void Histogram_Load(System::Object^ sender,
2 System::EventArgs^ e) {
3     test();
4 }
5
6 private: Void test()
7 {
8     chart1->Series["Dane"]->Points->Clear();
9
10    series->Points->AddXY(0, 1);
11    series->Points->AddXY(1, 3);
12    series->Points->AddXY(2, 2);
13    series->Points->AddXY(3, 5);
14
15    chart1->Invalidate();
16 }
17
```

Zwróć uwagę, że w powyższym *listningu* została zdefiniowana funkcja `test`, która uzupełnia wykres o przypadkowe dane (linie 10-13). Istotnym jest usunięcie wcześniej danych na wykresie – polecenie w linii 8. W celu wyświetlenia nowych danych należy wywołać polecenie z linii 15.

Na podstawie powyższego kodu uzyskamy następujący wynik działania programu:



W celu wyświetlenia informacji o substancjach w kubkach definiujemy nową funkcję, poniżej funkcji `test`:

```
1 private: Void update_subs_hist()
2 {
3     chart1->Series["Dane"]->Points->Clear();
4
5     std::vector<std::string> subs_name;
6     std::vector<int> vols;
7     std::vector<std::vector<int>> colors;
8     get_hist_data(&subs_name, &vols, &colors);
9
10    for (int i = 0; i < vols.size(); i++)
11    {
12        String^ name_cli = gcnew String(subs_name[i].c_str());
13        series->Points->AddXY(name_cli, vols[i]);
14        series->Points[i]->Color = Color::FromArgb(colors[i][0],
15        colors[i][1], colors[i][2]);
16    }
17    chart1->Invalidate();
18 }
19
```

Podobnie jak wcześniej, na początku usuwamy stare dane z wykresu (linia 3), a końcu wymuszamy wyświetlenie nowych danych (linia 17). W liniach 5-7 tworzymy pomocnicze wektory przechowujące dane do wyświetlenia. Zostają one uzupełnione w funkcji pomocniczej wywołanej w linii 8. Wewnątrz pętli for (linie 10-15) dodajemy nowe punkty do wykresu (linia 13) i zmieniamy kolor słupków wykresów na odpowiadający kolorom substancji (linia 14). Zwróć uwagę, że wartością zmiennej X jest w tym przypadku nazwa substancji (typ *String^*).

Kod funkcji uzupełniającej wektory o potrzebne dane (wstawiony na końcu pliku):

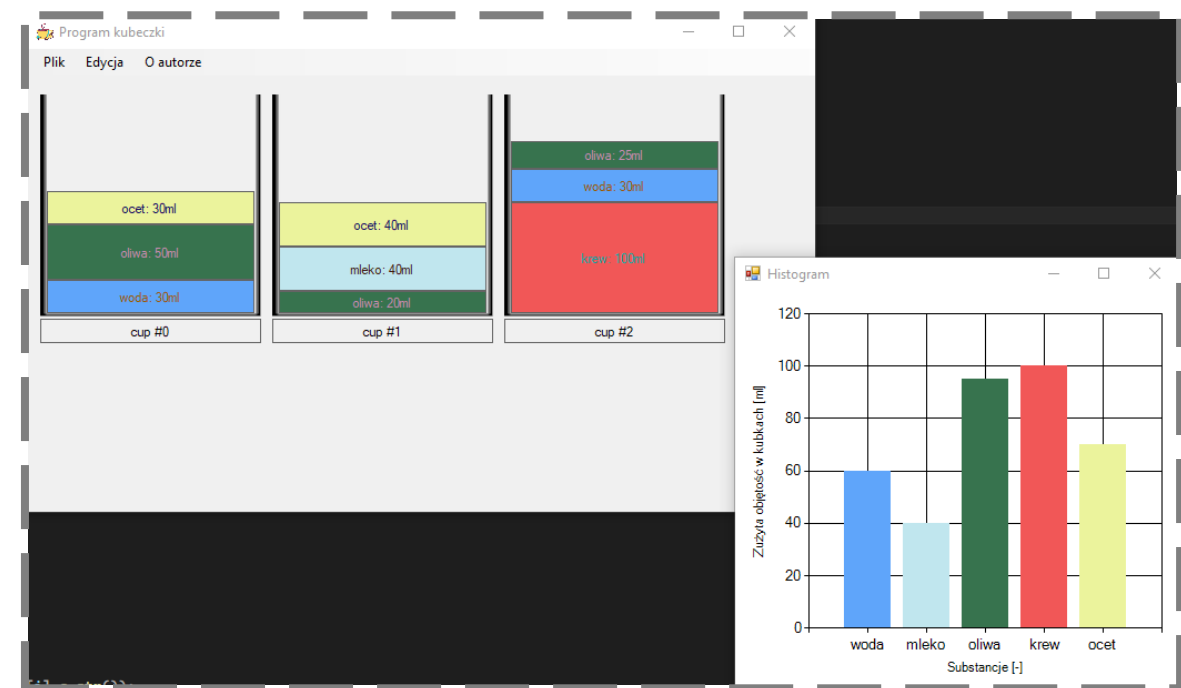
```
1 private: void get_hist_data(std::vector<std::string>* subs_name,  
2 std::vector<int>* vols, std::vector<std::vector<int>>* colors)  
3 {  
4     int cup_count = cups_pnt.size();  
5     int subs_count = substance_menu.size();  
6     for (int i_sub = 0; i_sub < subs_count; i_sub++)  
7     {  
8         std::string name = substance_menu[i_sub].get_name();  
9         std::vector<int> color = substance_menu[i_sub].get_color();  
10        (*subs_name).push_back(name);  
11        (*vols).push_back(0);  
12        (*colors).push_back(color);  
13  
14        for (int i_cup = 0; i_cup < cup_count; i_cup++)  
15        {  
16            int _id_in_cup = cups_pnt[i_cup]-  
17            >get_substance_id(name);  
18            if (_id_in_cup >= 0)  
19            {  
20                std::vector<double> cup_vols =  
21                cups_pnt[i_cup]->get_cup_volumes();  
22                (*vols)[i_sub] += cup_vols[_id_in_cup] *  
23                1e6;  
24            }  
25        }  
26    }
```

Pozostaje dodać odpowiednie referencje do plików i bibliotek:

```
1 #pragma once  
2 #include <vector>  
3 #include "TCup.h"  
4  
5  
6 namespace POiIOkubeczki {  
7
```

Sprawdź działanie programu!

Prześlij zmiany na repozytorium zdalne.

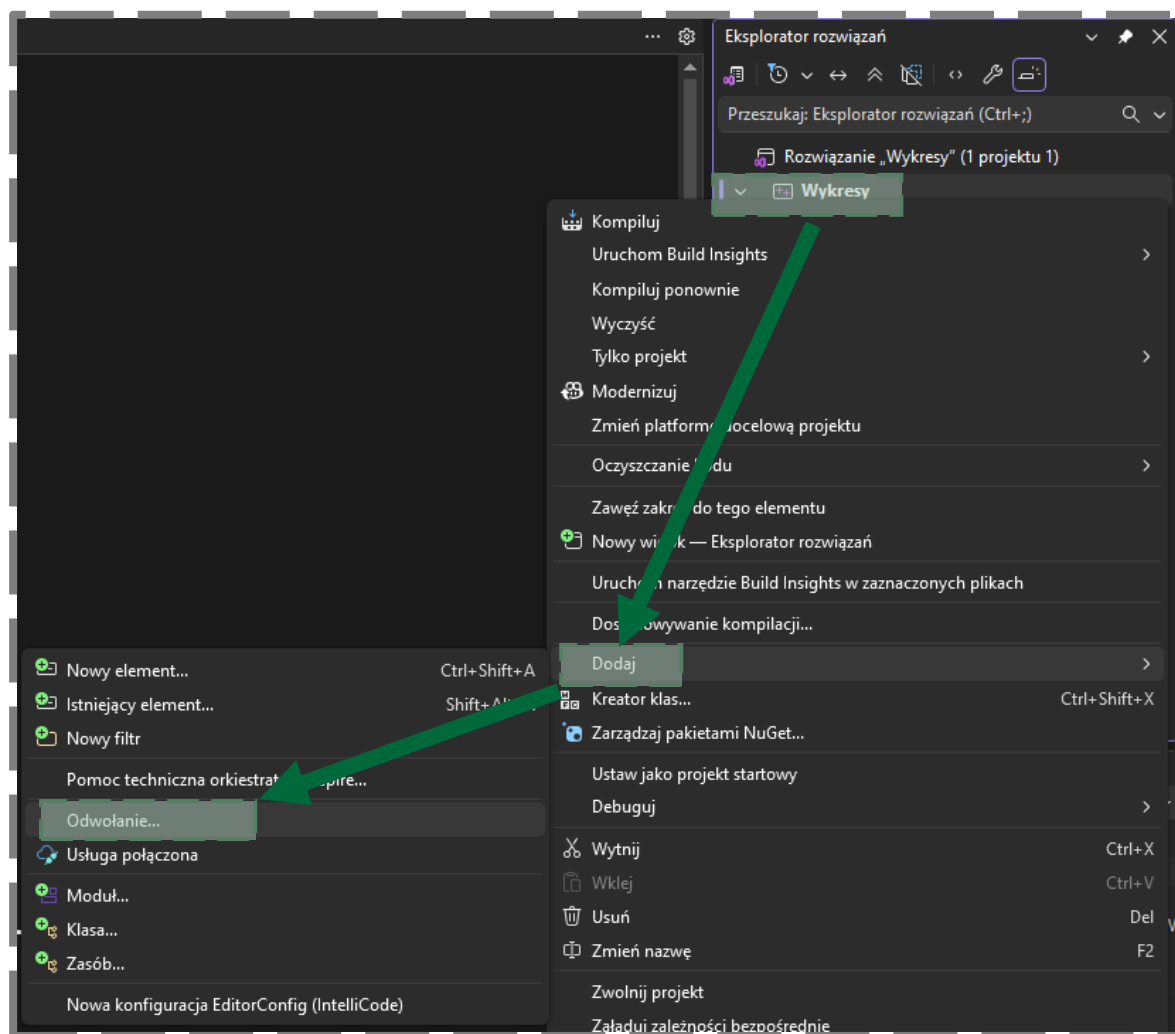


Na tym kończymy wspólną pracę nad aplikacją kubeczków.

ZADANIE 2.2 – wykres aktualizowany w czasie rzeczywistym

W celu pokazania dodatkowych funkcjonalności wykresów utworzymy nowy projekt. Wybierz szablon projektu „Pusty projekt CLR (.NET Framework)” w Visual Studio. W moim przypadku projekt nazwałem Wykresy.

Musimy pamiętać o dodaniu dynamicznej biblioteki do projektu. W tym celu możesz kliknąć PPM na projekt w drzewku, wybrać opcję *Dodaj* → *Odwołanie...*.



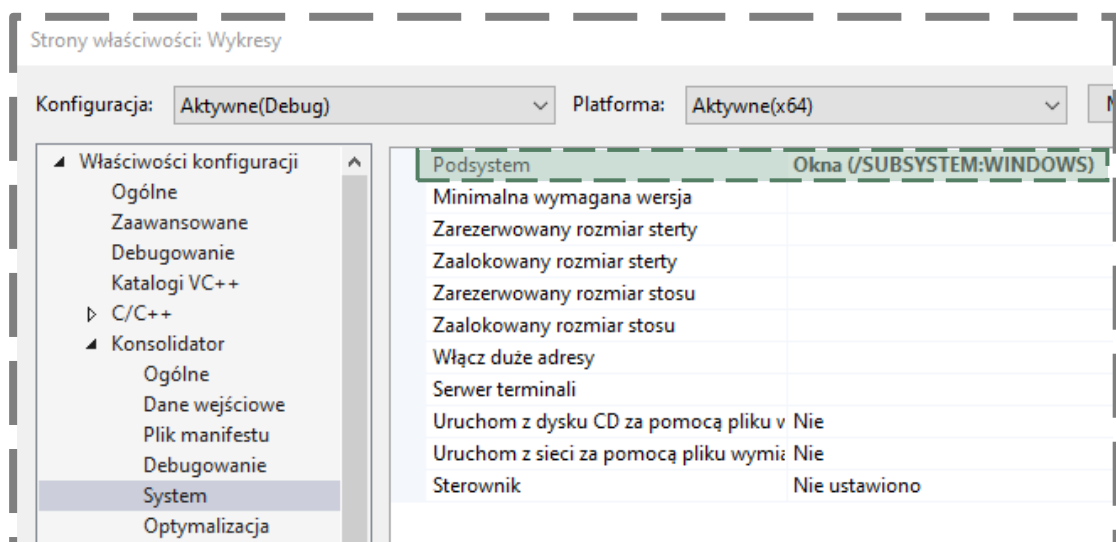
W nowo otwartym oknie należy przejść do *Zestawy->Framework* i zaznaczyć opcję *System.Windows.Forms.DataVisualization*.

Teraz utwórz nową *Formę*, w której wyświetlany będzie wykres. W tym celu kliknij PPM na nazwę projektu w jego drzewku i wybierz opcję *Dodaj->Nowy element...* W zakładce *Interfejs użytkownika* wybierz *Formularz systemu Windows* i wprowadź nazwę „Wykres.h”.

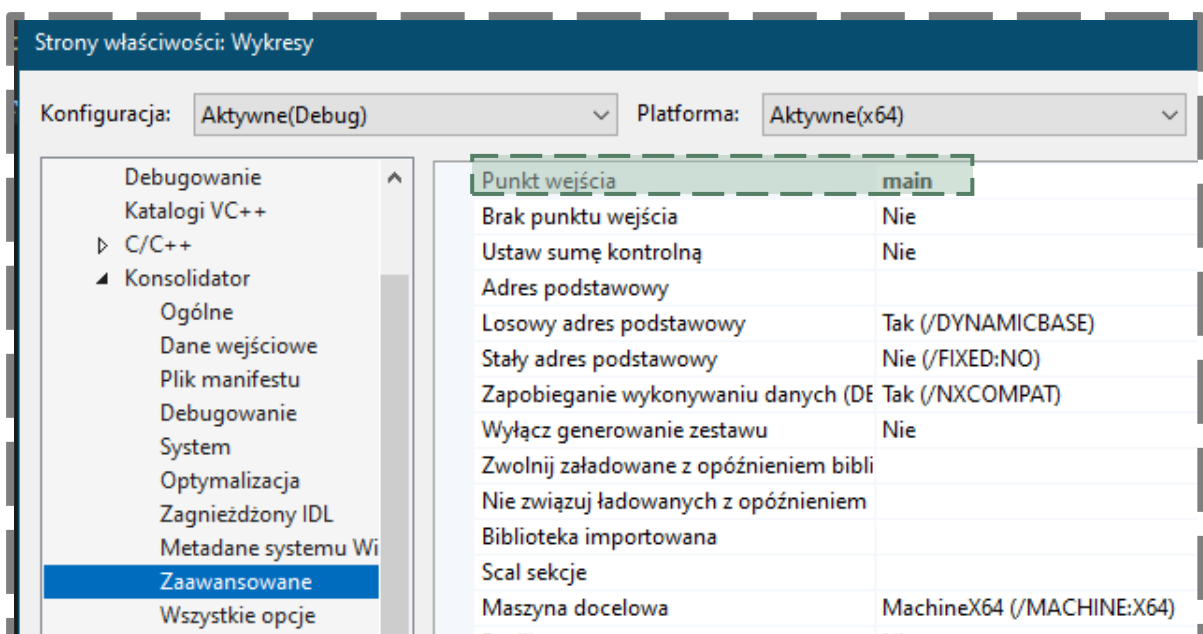
Uzupełniamy kod w pliku *Wykres.cpp*:

```
1 #include "Wykres.h"
2
3 using namespace System;
4 using namespace System::Windows::Forms;
5 [STAThreadAttribute]
6
7 int main(array<String^>^ args) {
8     Application::EnableVisualStyles();
9     Application::SetCompatibleTextRenderingDefault(false);
10    Wykresy::Wykres form;
11    Application::Run(% form);
12    return 0;
13 }
14
```

Należy pamiętać o zmianie właściwości projektów: kliknij PPM na nazwę projektu w drzewku i wybierz *Właściwości*. W nowo otwartym oknie przejdź do Konsolidator → System:



Następnie przejdź do zakładki Konsolidator → Zaawansowane:



Na tym etapie powinieneś już móc skompilować program – ujrzysz puste okno aplikacji.

W kolejnych krokach postępujemy identycznie jak w Zadaniu 1, musimy zdefiniować podstawowe zmienne i ustawienia wykresu.

Dodaj przestrzeń nazw:

```
1 namespace Wykresy {
2
3     using namespace System;
4     using namespace System::ComponentModel;
5     using namespace System::Collections;
6     using namespace System::Windows::Forms;
7     using namespace System::Data;
8     using namespace System::Drawing;
9     using namespace System::Windows::Forms::DataVisualization::Charting;
10 }
```

Zdefiniuj zmienne wykresu:

```
1 public ref class Wykres : public System::Windows::Forms::Form
2 {
3 private:
4     Chart^ chart1;
5     ChartArea^ area;
6     Series^ series;
7 }
```

Zdefiniuj funkcję inicjalizującą wykres – podkreślono nowe, dodane ustawienia względem zadania 1:

```
1 #pragma endregion
2 private: Void init_chart()
3 {
4     // create chart
5     chart1 = gcnew Chart();
6     chart1->Anchor = AnchorStyles::None;
7     chart1->Dock = DockStyle::None;
8     chart1->Location = System::Drawing::Point(12, 12);
9     chart1->Size = System::Drawing::Size(400, 409);
10    this->Controls->Add(chart1);
11    // create chart area
12    area = gcnew ChartArea("MainArea");
13    area->AxisX->Title = "Czas [s]";
14    area->AxisY->Title = "Temperatura [oC]";
15    area->AxisX->LabelStyle->Format = "0";
16    area->AxisX->Interval = 1;
17    chart1->ChartAreas->Add(area);
18    // create data series
19    series = gcnew Series("Dane");
20    series->ChartType = SeriesChartType::Line;
21    series->Color = Color::Red;
22    series->BorderWidth = 4;
23    series->ChartArea = "MainArea";
24    chart1->Series->Add(series);
25 }
26 }
```

Linie 6-9 służą do zdefiniowania stałego wymiaru wykresu i osadzenia go w oknie aplikacji. Linie 15-16 służą do poprawy wyświetlania wartości na osi X – bez części dziesiętnej. Linie 20-22 definiują wykres jako liniowy, z czerwoną pogrubioną linią.

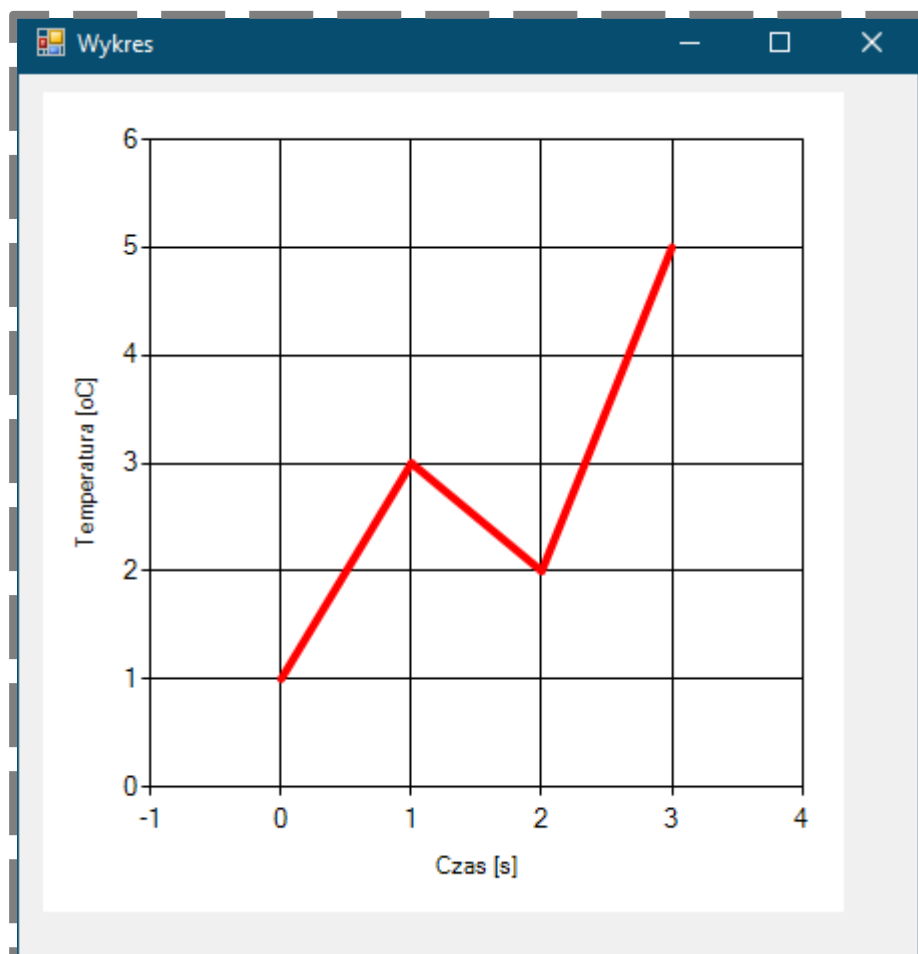
Wywołanie powyższej funkcji odbywa się w konstruktorze:

```
1 public:
2     Wykres(void)
3     {
4         InitializeComponent();
5         init_chart();
6     }
7
8
```

W IDE w zakładce „Wykres.h [Projekt]” kliknij dwukrotnie LPM na tło aplikacji. Zostaniesz przeniesiony do wygenerowanej funkcji, która wywoływana jest automatycznie w momencie otwarcia okna aplikacji. Uzupełnij jej kod:

```
1 private: void Wykres_Load(System::Object^ sender, System::EventArgs^
2 e) {
3     chart1->Series["Dane"]->Points->Clear();
4     series->Points->AddXY(0, 1);
5     series->Points->AddXY(1, 3);
6     series->Points->AddXY(2, 2);
7     series->Points->AddXY(3, 5);
8
9     chart1->Invalidate();
10 }
11
```

Po skompilowaniu programu powinieneś zobaczyć okno aplikacji z wykresem:



Naszym celem jest przetestowanie opcji automatycznej aktualizacji zawartości wykresu. Najprostszą metodą będzie skorzystanie z liczb pseudolosowych. U góry klasy zdefiniujemy dodatkowe zmienne:

```
1 public ref class Wykres : public System::Windows::Forms::Form
2 {
3 private:
4     Chart^ chart1;
5     ChartArea^ area;
6     Series^ series;
7
8     double time = 0;
9     Random^ rand = gcnew Random;
10    int val = 0;
11
```

Zmienna *rand* w linii 8 jest naszym generatorem liczb losowych. Zmienne *time* i *val* to odpowiednio wartości dla osi X i Y.

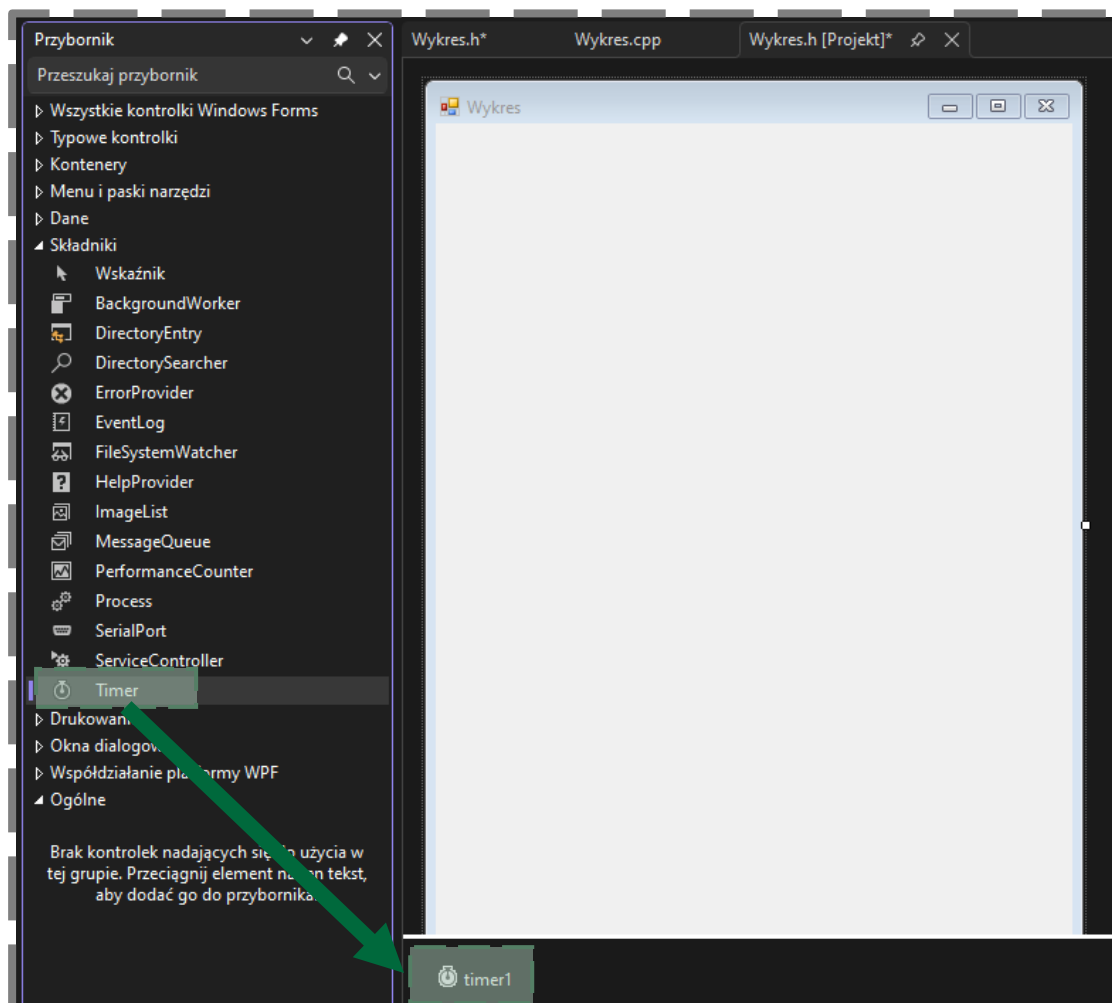
W funkcji *Wykres_Load* zdefiniujemy wartości początkowe zmiennych – podmieńmy kod funkcji:

```
1 private: System::Void Wykres_Load(System::Object^ sender, System::EventArgs^
2 e) {
3     chart1->Series["Dane"]->Points->Clear();
4
5     time = 0;
6     val = rand->Next(0, 300);
7
8     series->Points->AddXY(time, val);
9     chart1->Invalidate();
10 }
```

Zwróć uwagę na losowanie wartości z przedziału [0, 300] w linii 5.

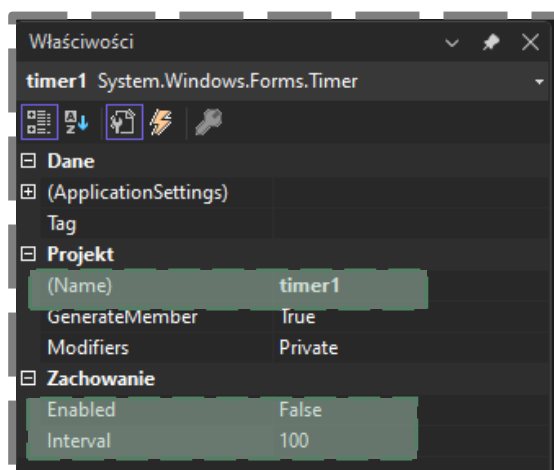
Potrzebujemy teraz narzędzia pozwalającego na dodawanie nowych punktów na wykresie w określonej częstotliwości. Do tego celu służy **timer**.

W *Przyborniku* kliknij dwukrotnie na kontrolę **Timer**, zostanie ona dołączona do projektu poniżej okna aplikacji:



Klikając LPM myszy na utworzony *timer* możesz zobaczyć jego opcje w oknie **Właściwości**:

- Domyślna nazwa kontrolki: „(Name) → *timer1*”.
- Licznik domyślnie jest wyłączony: „*Enabled* → *False*”.
- Wywołanie zdarzenia *timera* domyślnie co 100ms: „*Interval* → 100”.



Klikając tym razem dwukrotnie LPM na kontrolkę utworzonego *timera* zostaniesz przeniesiony do nowej, wygenerowanej funkcji obsługi zdarzenia. Funkcja ta będzie wywoływana co określony czas ustalony w opcji *Interval*. Uzupełnijmy jej kod:

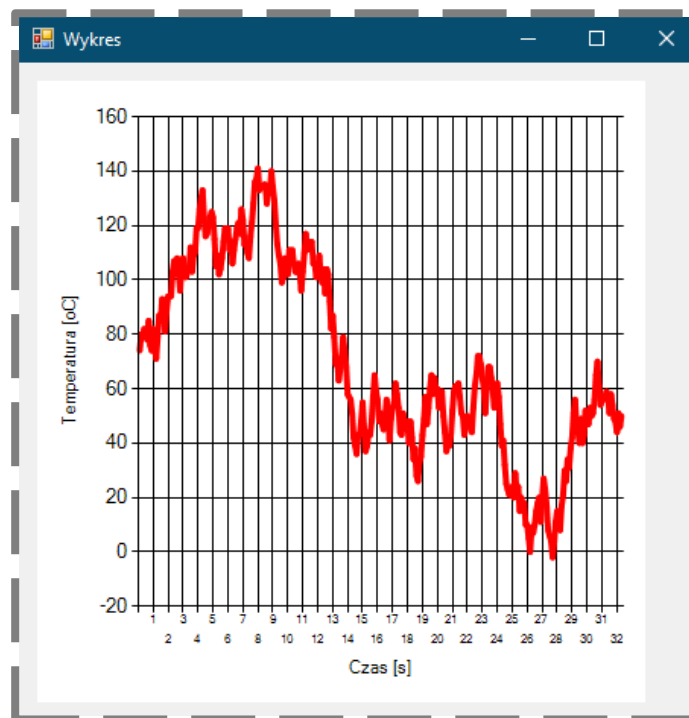
```
1 private: System::Void timer1_Tick(System::Object^ sender, System::EventArgs^
2 e) {
3     double dt = double(timer1->Interval) / 1000;
4     time += dt;
5
6     int d_val = rand->Next(-10, 10);
7     val += d_val;
8
9     series->Points->AddXY(time, val);
10    chart1->Invalidate();
11 }
12
```

W liniach 3-4 określamy czas od otwarcia okna aplikacji. W liniach 6-7 aktualizujemy wartość wyświetlaną na wykresie – ograniczamy możliwość zmiany wartości z poprzedniej chwili czasowej w zakresie [-10, 10]. W liniach 9-10 dodajemy nowy punkt do wykresu i odświeżamy jego widok.

Pozostaje jeszcze kwestia włączenia *timera* w momencie wczytania okna aplikacji:

```
1 private: System::Void Wykres_Load(System::Object^ sender, System::EventArgs^
2 e) {
3     chart1->Series["Dane"]->Points->Clear();
4
5     time = 0;
6     val = rand->Next(0, 300);
7
8     series->Points->AddXY(time, val);
9     chart1->Invalidate();
10    timer1->Enabled = true;
11 }
12
```

Sprawdź działanie programu!



Na koniec tego zadania zajmiemy się jeszcze ograniczeniem wyświetlanych danych do ostatnich 10sekund pomiaru. Dodatkowo zadbamy o automatyczne skalowanie się osi:

- osi X co wywołanie zdarzenia *timera*,
- osi Y co 1sekundę.

Zaktualizujemy kod funkcji obsługującej zdarzenie *timera*:

```
1 private: System::Void timer1_Tick(System::Object^ sender, System::EventArgs^
2 e) {
3     double dt = double(timer1->Interval) / 1000;
4     time += dt;
5     int d_val = rand->Next(-10, 10);
6     val += d_val;
7
8     series->Points->AddXY(time, val);
9
10    if (series->Points->Count > 10/dt) scaleX(dt);
11    if (time - int(time) < dt) scaleY();
12    chart1->Invalidate();
13 }
14
```

Na końcu pliku zaimplementujemy dwie nowe wywołane funkcje:

scaleX

```
1 private: Void scaleX(double dt)
2 {
3     series->Points->RemoveAt(0);
4     area->AxisX->Minimum = time - 10 - dt;
5     area->AxisX->Maximum = time + dt;
6 }
7
```


scaleY

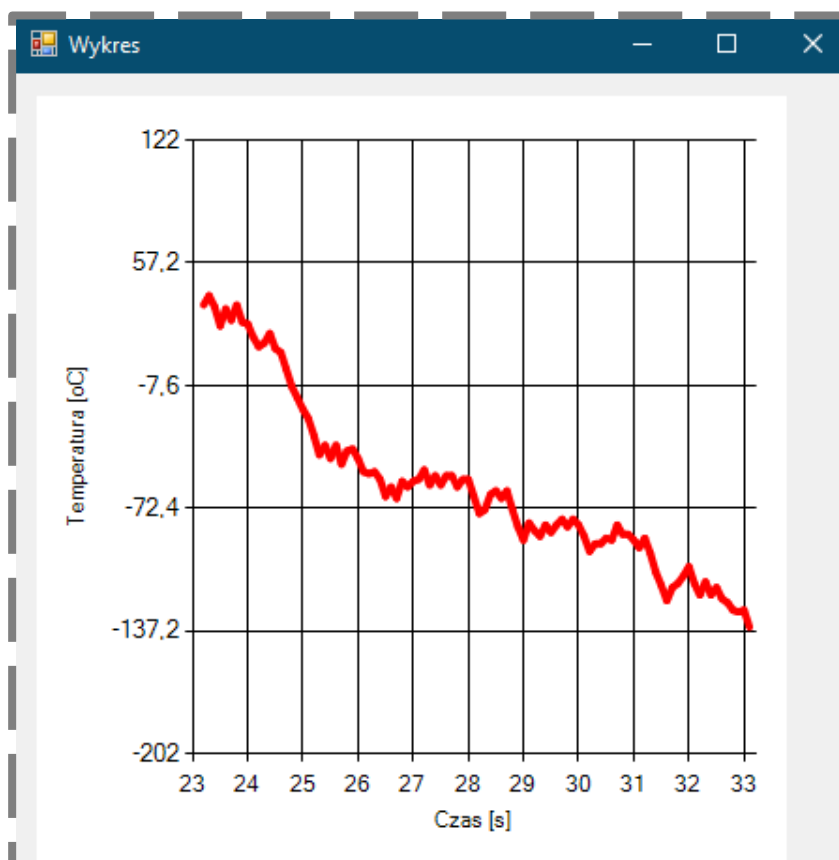
```
1 private: Void scaleY()
2 {
3     double min_y = series->Points[0]->YValues[0];
4     double max_y = min_y;
5
6     for each (DataPoint ^ p in series->Points)
7     {
8         double v = p->YValues[0];
9         if (v < min_y) min_y = v;
10        if (v > max_y) max_y = v;
11    }
12
13    double m = (max_y - min_y) * 0.5;
14    if (m == 0) m = 1;
15
16    area->AxisY->Minimum = min_y - m;
17    area->AxisY->Maximum = max_y + m;
18 }
19
```

W 3. linii listningu funkcji scaleX usuwany jest pierwszy punkt wykresu:

```
series->Points->RemoveAt(0);
```

Zdarzenie to ma miejsce w przypadku spełnienia instrukcji warunkowej w 10 linii kodu funkcji timer1_Tick. Pozostały kod funkcji scaleX i scaleY odpowiada za znalezienie wartości minimalnych i maksymalnych współrzędnych punktów wykresu.

Sprawdź działanie programu!



ZADANIE 2.3 – obsługa portu szeregowego

W celu pokazania prostoty inicjalizacji połączenia aplikacji z urządzeniem przez port szeregowy podłączyłem do PC drukarkę 3D – w tym przypadku Prusa MK3s (jest to o tyle istotne, że nie każda drukarka wspiera komunikację szeregową).

Na wstępie definiuję zmienną, która umożliwi mi dostęp do portu w całym zakresie klasy:

```
1 public ref class Wykres : public System::Windows::Forms::Form
2 {
3 private:
4     SerialPort^ serial;
5
6     Chart^ chart1;
7     ChartArea^ area;
8     Series^ series;
9
10    double time = 0;
11    int time_span = 120;
12    Random^ rand = gcnew Random;
13    double val = 0;
14
```

Dodatkowo, w linii 11, utworzyłem zmienną, która określa liczbę sekund pomiaru wyświetlanych na wykresie.

Należy jeszcze dodać odpowiednie przestrzenie nazw:

```
1 namespace Wykresy {
2
3     using namespace System;
4     using namespace System::ComponentModel;
5     using namespace System::Collections;
6     using namespace System::Windows::Forms;
7     using namespace System::Data;
8     using namespace System::Drawing;
9     using namespace System::Windows::Forms::DataVisualization::Charting;
10    using namespace System::IO::Ports;
11    using namespace System::Text;
12
```

Następnie definiuję podstawowe ustawienia komunikacji szeregowej:

```
1 #pragma endregion
2 private: Void init_serial()
3 {
4     serial = gcnew SerialPort();
5
6     serial->PortName = "COM7";
7     serial->BaudRate = 115200;
8     serial->Parity = Parity::None;
9     serial->DataBits = 8;
10    serial->StopBits = StopBits::One;
11
12    serial->Encoding = Encoding::ASCII;
13    serial->ReadTimeout = 500;
14    serial->WriteTimeout = 500;
15 }
16
```

Funkcję wywołuję w konstruktorze klasy:

```
1 public:
2     Wykres(void)
3     {
4         InitializeComponent();
5         init_chart();
6         init_serial();
7     }
8
```

Definiuję funkcję otwarcia portu:

```
1 private: void open_serial()
2 {
3     try
4     {
5         if (!serial->IsOpen)
6             serial->Open();
7     }
8     catch (Exception^ ex)
9     {
10        MessageBox::Show(ex->Message);
11    }
12
13    serial->DataReceived += gcnew SerialDataReceivedEventHandler(this,
14    &Wykres::serial_DataReceived);
15 }
```

Wewnątrz funkcji określono funkcję obsługi zdarzenia przestania informacji z urządzenia – implementuje kod funkcji na końcu pliku:

```
1 private: System::Void serial_DataReceived(System::Object^ sender,
2 System::IO::Ports::SerialDataReceivedEventArgs^ e) {
3     String^ received = serial->ReadLine();
4     this->Invoke(gcnew Action<String^>(this, &Wykres::update_val),
5     received);
6 }
```

Aplikacja działa dwuwątkowo, na osobnym wątku obsługiwana jest komunikacja szeregową. Dlatego w przypadku wywołania zdarzenia związanego z portem szeregowym koniecznym jest przestanie informacji z powrotem do wątku głównego – linia 3 i polecenie *Invoke*. W linii 3 pojawia się odwołanie do kolejnej funkcji, którą implementuje na końcu pliku:

```

1 private: Void update_val(String^ val_str)
2 {
3     if (val_str->Length > 22)
4     {
5         int _id_start = val_str->IndexOf("T");
6         int _id_end = val_str->IndexOf("/");
7         if (_id_start) val_str = val_str->Substring(_id_start + 2,
8 _id_end - 1);
9
10        int _id = val_str->IndexOf(".");
11        if (_id)
12        {
13            String^ val_int = val_str->Substring(0, _id);
14            String^ val_dec = val_str->Substring(_id + 1, 1);
15            val_str = val_int + "," + val_dec;
16        }
17
18        val_str = val_str->TrimEnd();
19
20        try {
21            val = Convert::ToDouble(val_str);
22        }
23        catch (...) {
24            MessageBox::Show("||" + val_str + "||", "Wykresy",
25 MessageBoxButtons::OK, MessageBoxIcon::Error);
26            val = 0;
27        }
28    }
29 }
30

```

Kod powyższej funkcji to jedynie próba odczytania z polecenia drukarki informacji o temperaturze dyszy. W przypadku pozyskania takiej informacji, wartość zmiennoprzecinkowa zostaje przypisana do zmiennej *val* – linia 21.

Wracamy do funkcji otwarcia portu i wywołujemy ją w momencie otwarcia okna aplikacji:

```

1 private: System::Void Wykres_Load(System::Object^ sender, System::EventArgs^
e) {
2     open_serial();
3
4     chart1->Series["Dane"]->Points->Clear();
5     chart1->Invalidate();
6
7     timer1->Enabled = true;
8 }
9

```

W funkcji obsługi zdarzenia wywołania *timer*a wysyłamy polecenie do drukarki z prośbą o odczyt temperatury dyszy:

```
1 private: System::Void timer1_Tick(System::Object^ sender, System::EventArgs^ e) {
2     serial->WriteLine("M105 T0\n");
3
4     double dt = double(timer1->Interval) / 1000;
5     time += dt;
6
7     if (val)
8     {
9         series->Points->AddXY(time, val);
10
11         if (series->Points->Count > time_span / dt) scaleX(dt);
12         if (time - int(time) < dt) scaleY();
13         chart1->Invalidate();
14     }
15 }
16
```

W oknie aplikacji dodajemy kontrolkę **NumericUpDown** umożliwiającą zmianę nastawy temperatury. Klikamy na nią dwukrotnie LPM w celu wygenerowania funkcji obsługującej zmianę wartości kontrolki – kod funkcji:

```
1 private: System::Void numericUpDown1_ValueChanged(System::Object^ sender,
2 System::EventArgs^ e) {
3     Decimal tmp = numericUpDown1->Value;
4     int temp = Convert::ToInt16(tmp);
5
6     if (temp < 280)
7     {
8         serial->WriteLine("M104 S" + temp + "\n");
9         MessageBox::Show("Ustalono nową temperaturę dyszy: " + temp +
10 "stC w " + time + "sekundzie pomiaru", "Wykresy", MessageBoxButtons::OK,
11 MessageBoxIcon::Error);
12     }
13 }
14
```

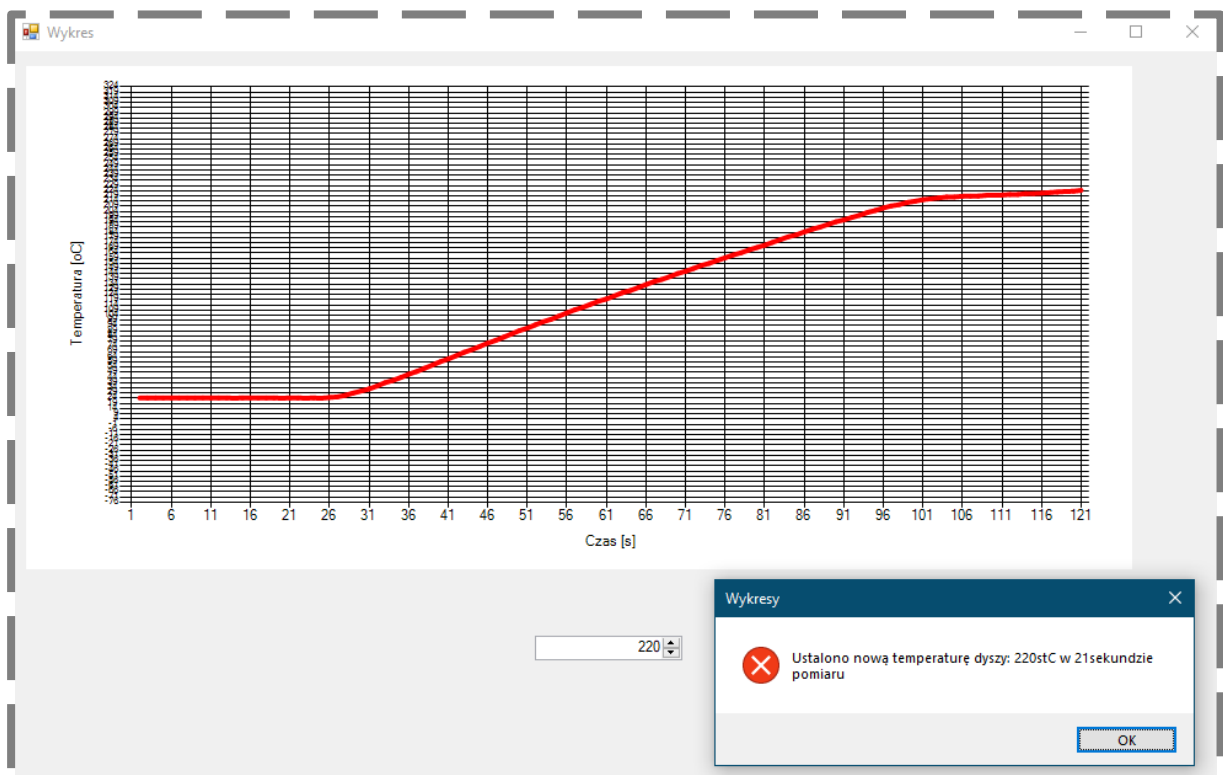
W funkcji korzystamy ze zmiennej *time_span*:

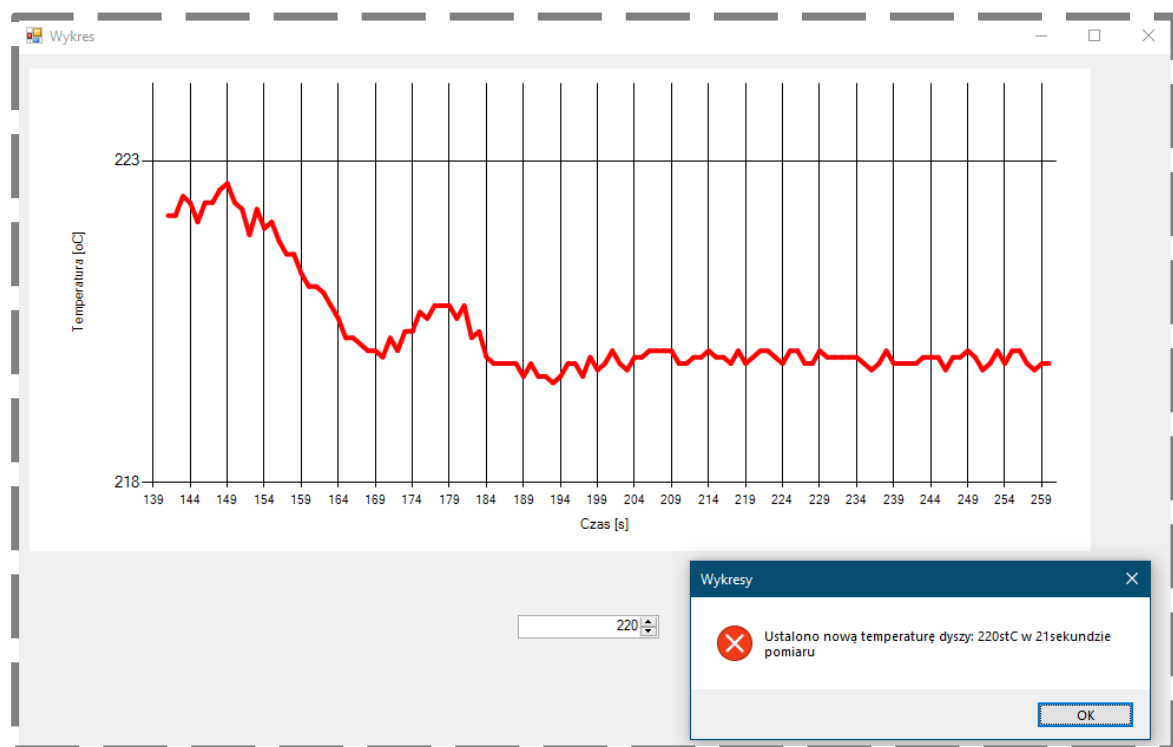
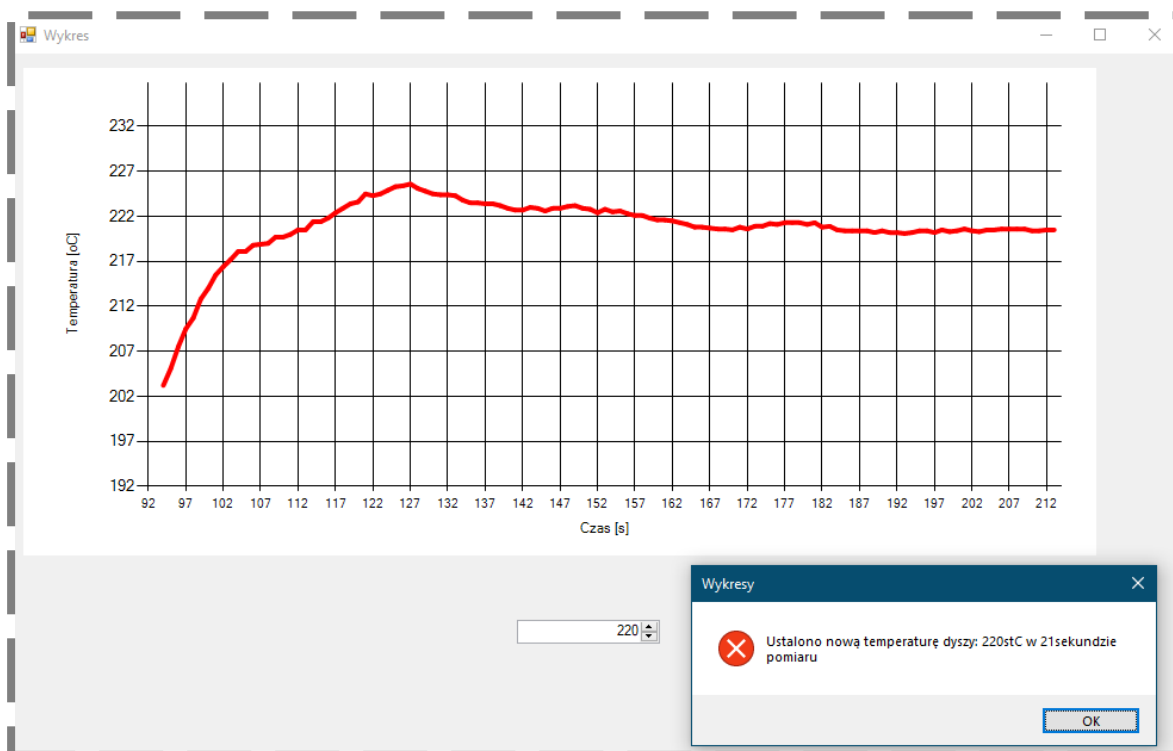
```
1 private: void scaleX(double dt)
2 {
3     series->Points->RemoveAt(0);
4     area->AxisX->Minimum = time - time_span - dt;
5     area->AxisX->Maximum = time + dt;
6 }
7
```

Zmieniamy nieznacznie parametry wykresu:

```
1 private: Void init_chart()
2 {
3     // create chart
4     chart1 = gcnew Chart();
5     chart1->Anchor = AnchorStyles::None;
6     chart1->Dock = DockStyle::None;
7     chart1->Location = System::Drawing::Point(12, 12);
8     chart1->Size = System::Drawing::Size(900, 409);
9     this->Controls->Add(chart1);
10    // create chart area
11    area = gcnew ChartArea("MainArea");
12    area->AxisX->Title = "Czas [s]";
13    area->AxisY->Title = "Temperatura [oC]";
14    area->AxisX->LabelStyle->Format = "0"; // bez części dziesiętnej
15    area->AxisX->Interval = 5;
16    area->AxisY->LabelStyle->Format = "0"; // bez części dziesiętnej
17    area->AxisY->Interval = 5;
18    chart1->ChartAreas->Add(area);
19    // create data series
20    series = gcnew Series("Dane");
21    series->ChartType = SeriesChartType::Line;
22    series->Color = Color::Red;
23    series->BorderWidth = 4;
24    series->ChartArea = "MainArea";
25    chart1->Series->Add(series);
26 }
27
```

Efekt działania programu:





3. Podsumowanie

Na tej instrukcji kończymy cykl zajęć zapoznawczych z językiem C++/CLI. Połączenie języka z jego konfiguracją w Visual Studio dostarcza wiele przydatnych narzędzi, które w łatwy sposób można wykorzystać w swoich projektach. W niniejszej instrukcji pokazano przykłady tworzenia wykresów oraz komunikacji szeregowej z zewnętrznym urządzeniem, które, mam nadzieję, okażą się pomocne na dalszym etapie studiów.